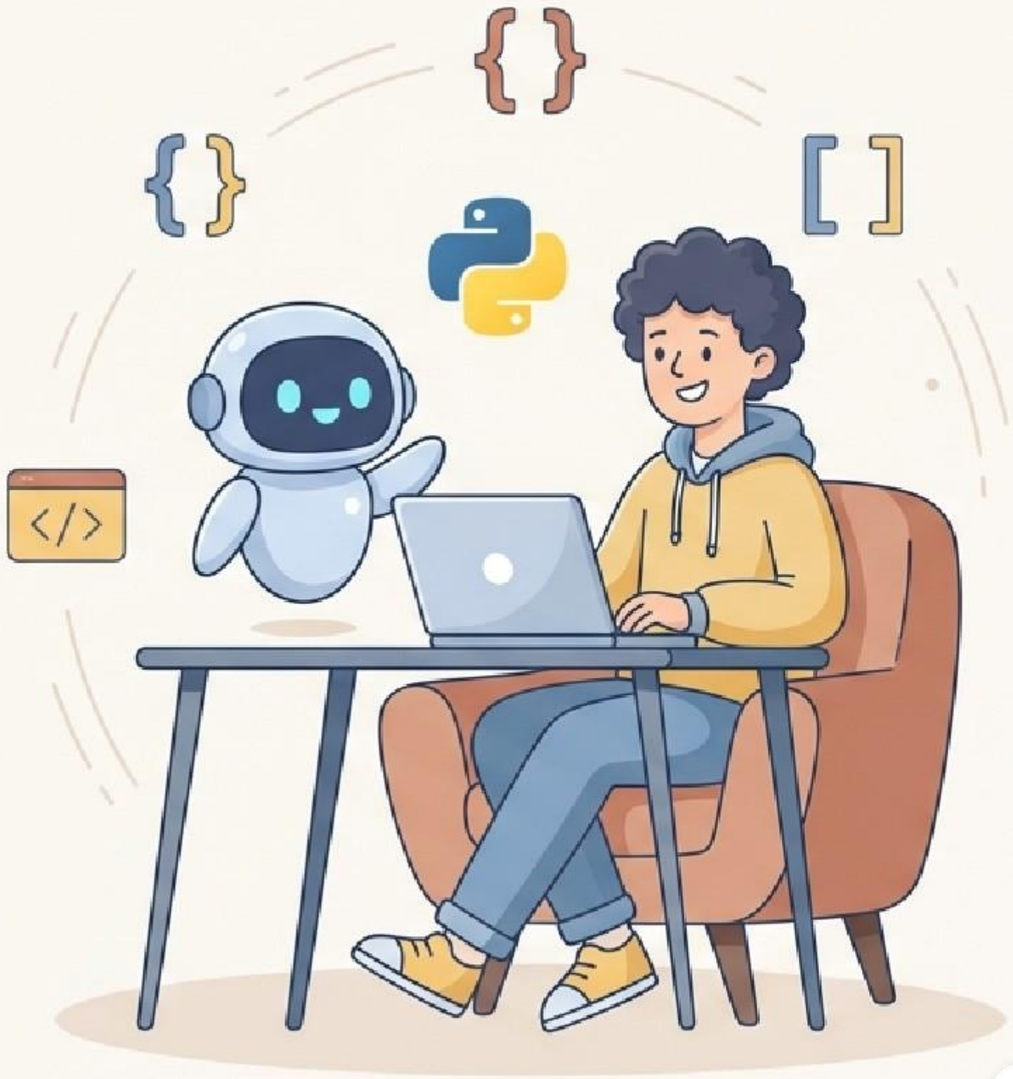
أولى خطواتك في البرمجة وبايثون

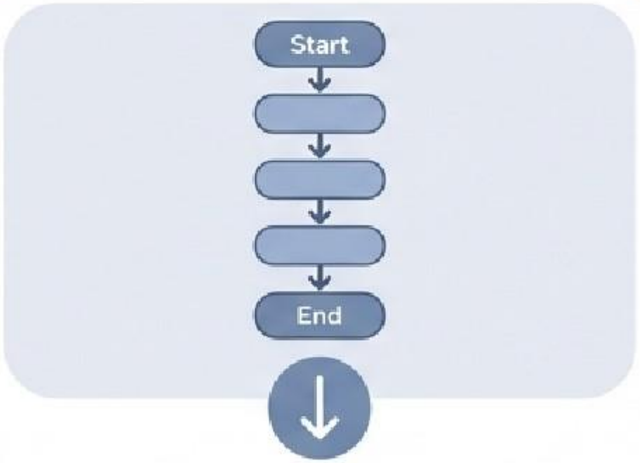
ملخص مبسط وسريع.. كأنك بتذاكر مع صاحبك على القهوة لأول مرة! ☕💻

المحاضرة الثانية



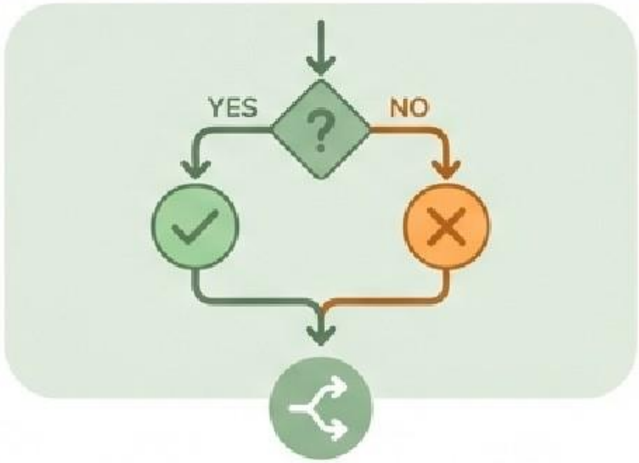
إزاي نفكر زي الكمبيوتر؟ (خرائط التدفق)

- عشان تحل أي مشكلة برمجية، لازم ترتب أفكارك الأول. هنا بيجي دور خرائط التدفق (Flowcharts).
- ببساطة: هي رسمة بتوضح خطوات حل المشكلة بالترتيب عشان الكمبيوتر يفهمها.
- أنواعها الأساسية: التتابع البسيط، القرارات/الفروع، والتكرار/الدورانات.



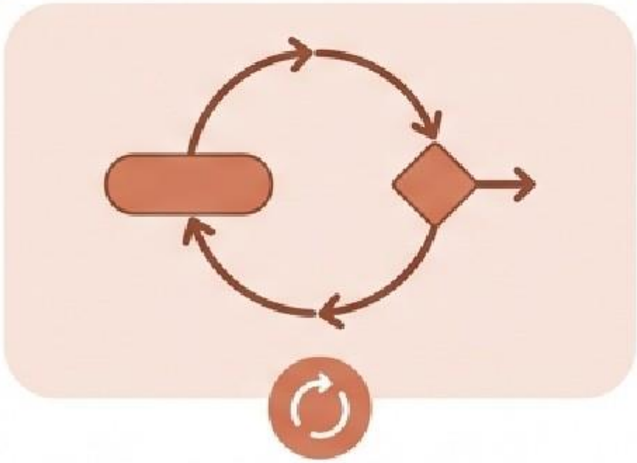
التتابع البسيط

خطوات متتالية بالترتيب دون أي شروط.



القرارات أو الفروع

اختيار مسار بناءً على شرط معين (نعم/لا).



التكرار أو الدورانات

تكرار مجموعة من الخطوات حتى يتحقق شرط معين.

مثال من حياتنا: المنبه الصبح ⏰

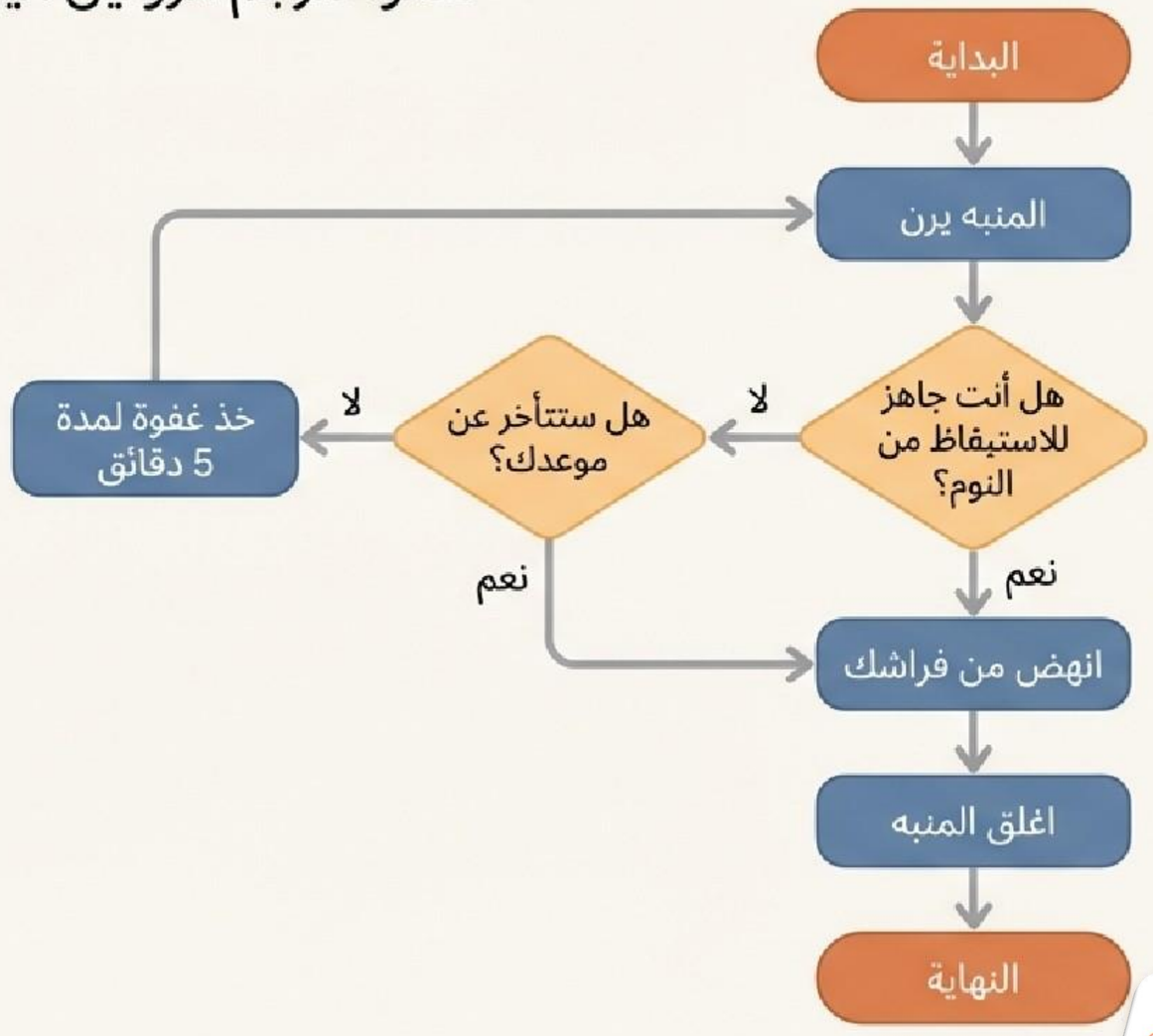
تعالوا نترجم الروتين اليومي لخريطة تدفق!

- البداية ➡️ المنبه يرن ⬅️ سؤال: جاهز تصحى؟

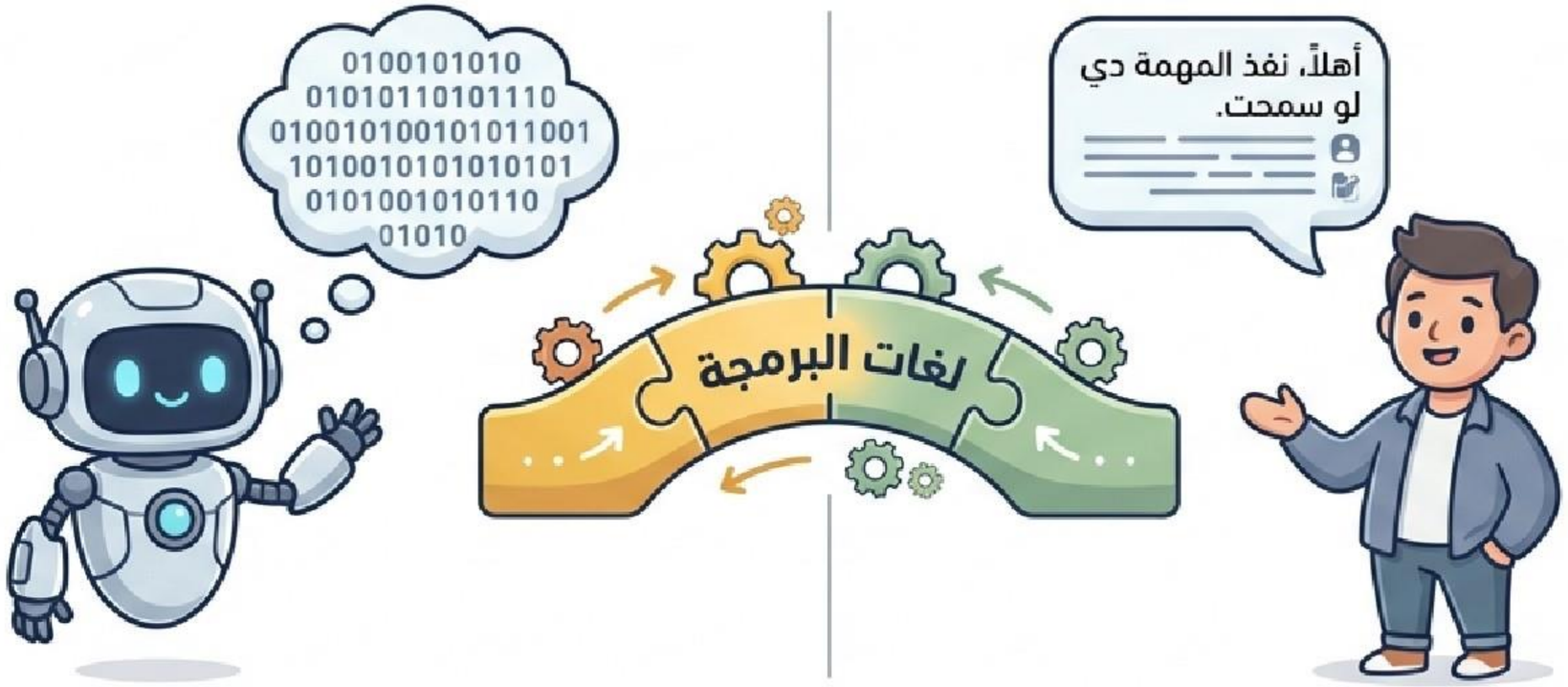
✅ لو آه: قوم من السرير واقفل المنبه (النهاية).

❓ لو لأ: سؤال تاني: هتتأخر على ميعادك؟

🚶‍♀️ لو آه: قوم فوراً! / 🚶‍♀️ 🕓 لو لأ: خد غفوة 5 دقايق وارجع لخطوة المنبه. (ده مثال على التفرع والدوران!).



يعني إيه برمجة أصلاً؟ وليه محتاجينها؟

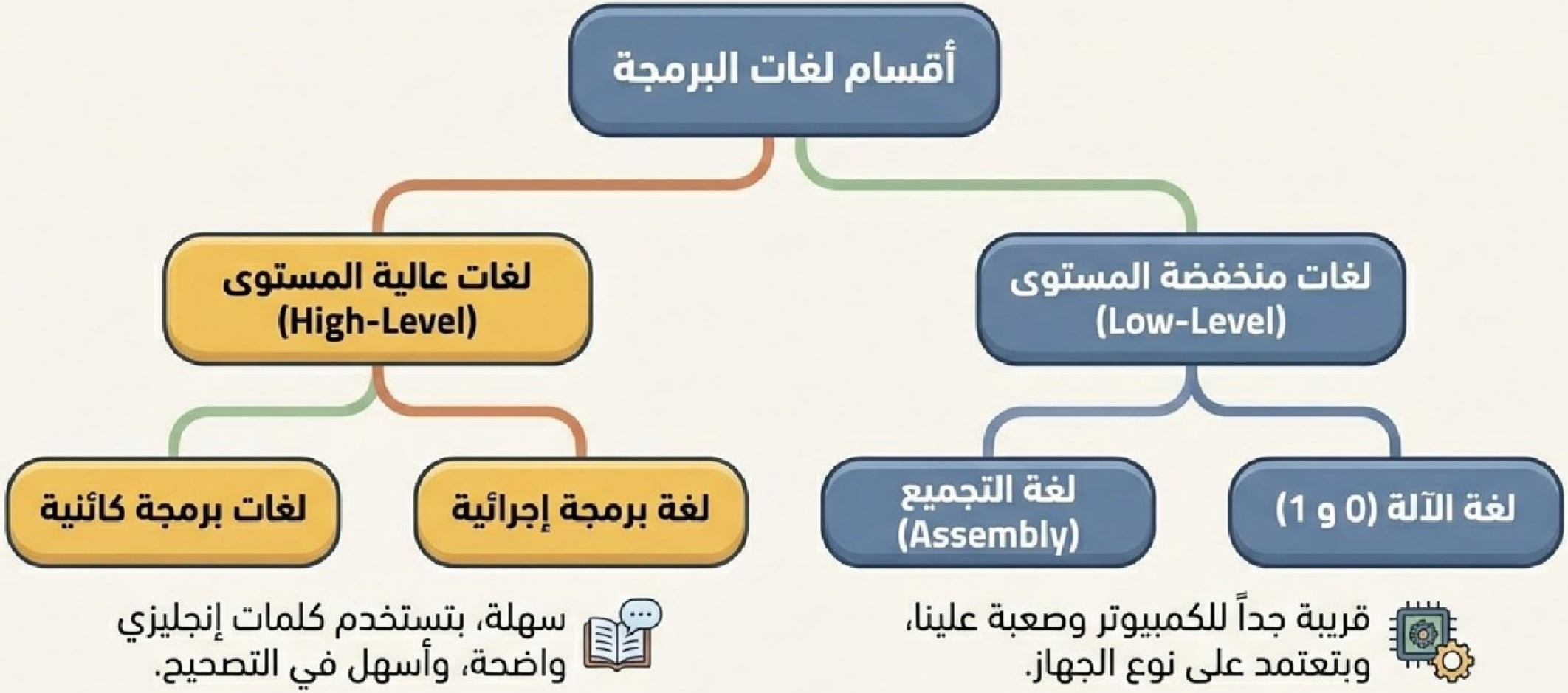


البرمجة: هي إنك تدي أوامر للكمبيوتر عشان ينفذ مهمة معينة بطريقة محددة.

المشكلة: الكمبيوتر مبيفهمش غير لغة الآلة (Machine Language)، ودي عبارة عن أصفار ووحايد 0 و 1 بس! 🤖

الحل: لغات البرمجة! دي الكوبري اللي بيننا وبين الكمبيوتر. كل سطر بنكتبه اسمه كود (Code) أو تعليمة.

شجرة عيلة لغات البرمجة



إزاي الكود بيتحول لأصفار ووحايد؟

- عشان الكمبيوتر يفهم اللغات العالية، الكود لازم يترجم!
- كل لغة برمجة ليها أداة خاصة بيها بتعمل المهمة دي.
- الأداة دي إما بتكون **مترجم** (Compiler) أو **مفسر** (Interpreter).

- **كل لغة برمجة ليها أداة** خاصة بيها بتعمل المهمة دي.
- الأداة دي إما بتكون **مترجم** (Compiler) أو **مفسر** (Interpreter).
- الاتنين بيعملوا نفس الحاجة بس بأسلوب مختلف.. تعالوا نشوف الفرق!

المترجم VS المفسر

المترجم (Compiler)	المفسر (Interpreter)
طريقته: بيحلل البرنامج كله على بعضه ويترجمه مرة واحدة.	**طريقته:** بترجم الكود سطر سطر.
السرعة: سريع جداً في التنفيذ.	**السرعة:** أبطأ شوية في التنفيذ.
الأخطاء: بيكتشف الأخطاء بعد تحليل البرنامج كامل.	**الأخطاء:** سهل جداً تتبع الأخطاء لأنه بيقف فوراً عند السطر الغلط.
أمثلة اللغات: C, C++, C#	**أمثلة اللغات:** Python, PHP, Ruby

ورشة عمل المبرمج: خطوات تنفيذ البرنامج 🛠️

عشان تنفذ أي برنامج محتاج 4 أدوات:

المترجم/المفسر

اللي بيحول الكود للغة الآلة عشان الكمبيوتر الكمبيوتر يفهمه.

المحرر (Editor)

المكان اللي بنكتب فيه الكود (زي VS Code أو Anaconda).

أدوات تصحيح الأخطاء (Debuggers)

بتساعدك تكتشف الثغرات وتصلحها.

أنظمة التحكم في الإصدارات

بتحفظ التعديلات وبتسمحلك ترجع لنسخ قديمة (أشهرها GitHub).

نجم الساحة.. لغة بايثون 🐍 Python

Guido Van Rossum

- 🐍 لغة برمجة عالية المستوى، ابتكرها Guido Van Rossum سنة 1991.

🐍 ليه بايثون مميزة جداً؟

- ✅ مفتوحة المصدر (Open Source) ومجانية تماماً!
- ✅ بتشتغل على أي نظام تشغيل في العالم.
- ✅ بتدعم البرمجة كائنية التوجه (OOP) زي الكلاس، الكائن، التغليف، والوراثة.

إزاي نتعلم بايثون؟ (زي ما اتعلمنا العربي بالظبط!)

خطوات تعلم اللغة العربية:

الحروف ← الكلمات ← الجمل ← الفقرة

خطوات تعلم لغة البرمجة بايثون:

الرموز والأرقام والحروف ← الكلمات المفتاحية (Keywords) ← التعليمات (Instructions) ← برنامج كامل (Program)

الكلمات المفتاحية **(Keywords)** هي كلمات محجوزة في لغة بايثون ليها معنى ثابت.
(بايثون 3.11 فيها 36 كلمة مفتاحية زي `if, for, class`).

إتيكيت وقواعد كتابة الكود في بايثون 📜

المسافات البادئة (Indentation):
أهم حاجة في بايثون!
المسافات في أول السطر بتحدد بنية الكود ومين تابع لمين.

سطر لكل تعليمة:
كل أمر برمجي يتكتب في سطر لوحده بالترتيب اللي عايزه عايزه يتنفذ بيه.

فراغات للتوضيح:
ممكن تسيب مسافات جوه السطر نفسه عشان الكود يكون مريح للعين.

مفيش فواصل منقوطة (;):
بايثون لغة نظيفة، مش محتاج تحط ; في آخر كل سطر زي اللغات التانية.

```
def greet(name):
    print("مرحبًا, " + name + "!")
    print("كيف حالك اليوم")
```

صناديق الذاكرة: المتغيرات والثوابت 📦

المتغير (Variable)

عامل زي صندوق في ذاكرة الكمبيوتر، بنسميه اسم معين ونشيل جواه داتا (قيمة).

من اسمه، نقدر نغير القيمة دي في أي وقت أثناء تشغيل البرنامج.

الثابت (Constant)

قيمة ثابتة مستحيل تتغير طول ما البرنامج البرنامج شغال.

قواعد تسمية المتغيرات.. المسموح والممنوع ✅🚫

المسموح ✅

- طول الاسم من 1 إلى 31 حرف.
- حروف صغيرة/كبيرة، أرقام، وعلامة (Underscore). a A 1 _
- الشرط الأول والأهم: لازم تبدأ بحرف أو _. ممنوع تبدأ برقم!

الممنوعات 🚫

- مفيش مسافات جوه الاسم.
- ممنوع استخدام أي رموز غريبة (زي @, !, #). # ! @ % &

نصائح المحترفين: ابدأ بحرف صغير، خلي الاسم بيعبر عن المحتوى (زي salary)، ولو الاسم كلمتين استخدم الـ **mixedCase** أو افصلهم بـ _.

اختبر نفسك: صح ولا غلط؟ 🤔

أي الأسماء دي تنفع لمتغير في بايثون؟

✅ `salary=20`
(صح - حروف فقط)

❌ `2salary=30`
(غلط - بيبدأ برقم)

❌ `'salary=30`
(غلط - بيبدأ برمز غريب)

✅ `_salary=20`
(صح - بيبدأ بـ Underscore)

❌ `!salary=20`
(غلط - بيحتوي على رمز ممنوع)

خريطة أنواع البيانات (Data Types)

الأعداد (Numbers)

- **صحيحة Int** (مثال: 3)
- **عشرية Float** (مثال: 1.5)
- **مركبة Complex** (مثال: 4J)

نصوص (String)

أي حروف أو كلمات بين علامات علامات تنصيص (مثال: 'hello')

منطقية (Boolean)

إجابات آه أو لأ (True أو False)

مجموعات متقدمة

زي القوائم (Lists) والقواميس (Dictionaries)

مبروك! أول برنامج ليك في الحياة 🎉

هنتعلم نكتب كود في ملف اسمه first_program.ipynb (على برنامج Jupyter).

first_program.ipynb

```
In [1]: print("اهلا بالعالم")
```

```
اهلا بالعالم
```

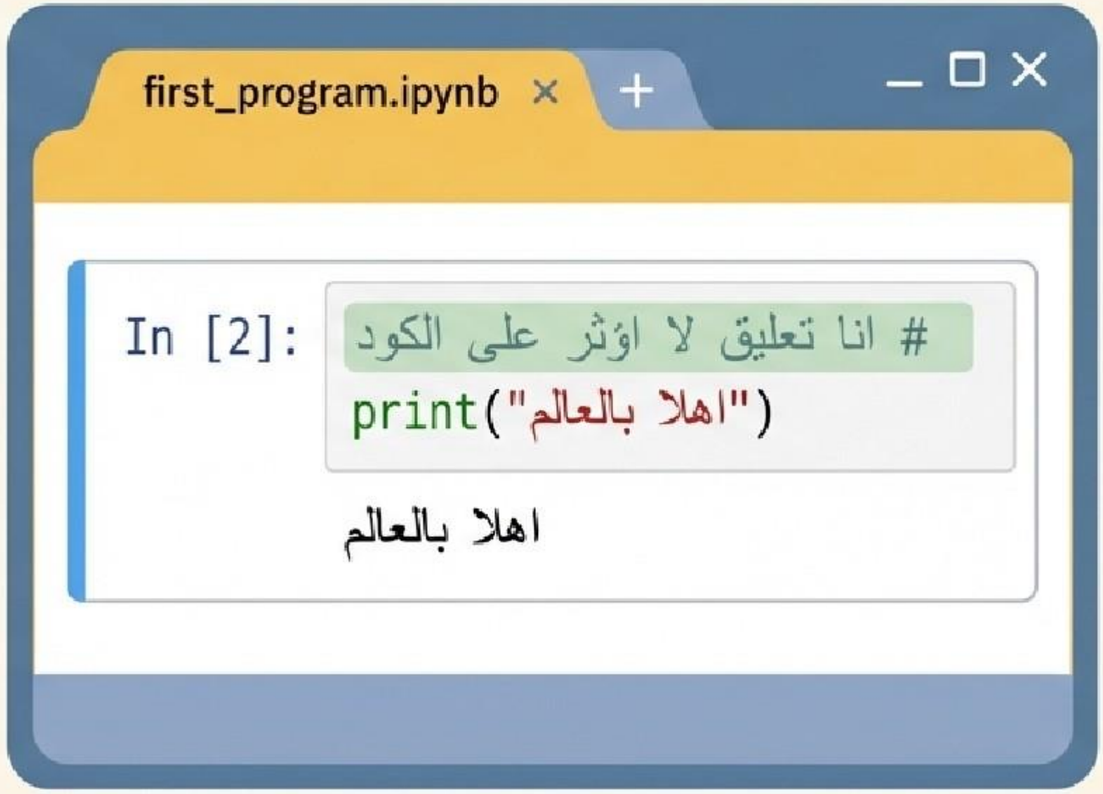
هنستخدم دالة اسمها print(). دي دالة دالة مدمجة (Built-in) جاهزة في بايثون بتطبع أي حاجة تديها لها على الشاشة.

إزاي نكتبها؟
بنفتح قوسين، ونحط النص بتاعنا بين علامتين تنصيص عشان بايثون تفهم إن ده نص (String).

التعليقات (Comments): اتكلم مع نفسك جوه الكود 🤫

⚠️ الحل: نستخدم رمز الشباك # قبل الكلام.

👻 أي حاجة بتيجي بعد الـ # بايثون بتعمل نفسها مش شايفاها ومبتتأثرش بيها خالص أثناء التشغيل!



💬 أحياناً بنحتاج نكتب ملحوظة لنفسنا أو لأي مبرمج تاني هيقرا الكود بعدنا عشان نفتكر الكود ده بيعمل إيه.

#️⃣ لو كتبنا كلام عادي، بايثون هتفتكره كود وتديك خطأ (Error).

حصة الحساب في بايثون 🧮

بايثون بتعمل كل العمليات الحسابية بسهولة جداً:

- الجمع (+)
- الطرح (-)
- الضرب (*)
- القسمة (/)

الجديد بقى: علامة % (Modulus). دي مش معناها نسبة مئوية!
دي بتجيب باقي القسمة.
(مثال: 11 % 3 بيساوي 2)

ترتيب العمليات: مين بيسبق مين؟ 🚦

لو عندنا مسألة طويلة، بايثون بتحلها بالترتيب ده (من الأعلى للأدنى):

1. **الأقواس ( )** - دايماً بتكسب وتتنفذ الأول
2. **الأسس **** - ضرب الرقم في نفسه
3. الضرب *، القسمة / و //، وباقي القسمة %
4. الجمع + والطرح -

ملحوظة: لو العمليات في نفس المستوى (زي ضرب وقسمة مع بعض)، بنحل من اليسار لليمين.

أمثلة سريعة.. تعالوا نحسبها سوا! 📝

```
print(10 - 4 + 3)
```

- نفس المستوى، نمشي من اليسار لليمين:
  10 − 4 = 6
- ثم نجمع:
  6 + 3

الناتج = 9

```
print(20 / 5 * 2)
```

- نفس المستوى، نمشي من اليسار لليمين:
  20 ÷ 5 = 4
- ثم نضرب:
  4 × 2

الناتج = 8

```
print(5 + 3 * 2)
```

- الضرب الأول:
  3 × 2 = 6
- ثم الجمع:
  6 + 5

الناتج = 11

عاش جدًّا! 🚀

✅ كده إنت حطيت رجلك على أول الطريق.

💡 فهمت إزاي الكمبيوتر بيفكر، عرفت أدواتك، وكتبت أول كود ليك بلغة بايثون.

⌨️ البرمجة كلها ممارسة وتجربة.. متخافش تغلط عشان دي أكتر طريقة هتعلمك.

🌟 بالتوفيق في رحلتك الجاية يا مبرمج!